Synopsis

The AI-Powered Zoho Project Chatbot is a conversational system that enables users to interact with their Zoho Projects workspace using natural language. The system integrates with Zoho's REST APIs and allows users to perform both read and write operations such as viewing projects, managing tasks, and analyzing workload distribution.

The application uses OAuth 2.0 Authorization Code Grant flow to securely authenticate users with their own Zoho accounts, ensuring no shared credentials. Once authenticated, users interact with a chat interface where their requests are processed by a multi-agent backend system built using LangGraph.

The system consists of two specialized agents:

- A Query Agent for retrieving data (projects, tasks, users)
- An Action Agent for modifying data (create, update, delete), with mandatory human confirmation

A router (supervisor) intelligently directs user queries to the appropriate agent. The chatbot maintains both short-term memory (within a session) and long-term memory (across sessions), enabling contextual and personalized interactions.

Tech Stack

Backend

- FastAPI – API framework for building async endpoints
- Python – Core programming language
- LangGraph – Multi-agent orchestration and stateful workflow
- Pydantic – Request/response validation
- HTTPX / Requests – API communication with Zoho

Authentication

- OAuth 2.0 (Authorization Code Grant) – Secure user authentication
- Zoho OAuth Credentials

Frontend

- React / Next.js – Chat UI

Database / Storage

- SQLite (for:
    - User tokens
    - Long-term memory
)

AI / NLP

- LLM (OpenAI / Gemini or similar)
- Prompt-based routing and reasoning

Other Tools

- dotenv – Environment configuration

System Architecture

The system follows a modular, multi-agent architecture with clear separation of concerns.

High-Level Flow

1. User logs in via Zoho OAuth
2. Backend stores access & refresh tokens
3. User sends a chat message
4. Router decides which agent to invoke
5. Agent processes request using tools
6. If action → requires confirmation (HIL)
7. Response returned to UI

Core Components

1. Frontend (Chat UI)

- Displays conversation thread
- Handles login redirect
- Sends user input to /chat endpoint

2. FastAPI Backend

- /auth/login → Redirect to Zoho OAuth
- /auth/callback → Handle token exchange
- /chat → Main chatbot endpoint
- Middleware → Identify authenticated user

3. Router (Supervisor Node)

- Analyzes user intent
- Routes request to:
    - Query Agent (read)

    - Action Agent (write)

4. Agents

Query Agent

- Handles:
    - list_projects
    - list_tasks
    - get_task_details
    - list_project_members
    - get_task_utilisation

Action Agent

- Handles:
    - create_task
    - update_task
    - delete_task
- Includes **Human-in-the-Loop confirmation**

5. Tools Layer (8 Tools)

- Each tool maps to a Zoho API function
- Encapsulated in reusable functions/classes

6. Memory System

- Short-term memory: session-based context
- Long-term memory: stored in DB (preferences, history)

7. Zoho API Client

- Handles:
    - API calls
    - Token refresh
    - Request formatting

System Requirements

Functional Requirements

- Zoho OAuth 2.0 user authentication
- Chat-based interface for interacting with Zoho Projects

- Multi-agent system (Query + Action) with routing
- Support for all 8 required tools (projects, tasks, users, etc.)
- Human-in-the-loop confirmation for all write operations
- Session-based and persistent memory support

Non-Functional Requirements

- Asynchronous backend (FastAPI)
- Secure token handling and storage
- Modular and maintainable architecture
- Responsive and user-friendly chat UI

Software Requirements

- Python 3.9+
- Node.js (for frontend)
- Web browser

Hardware Requirements

- Minimum 4 GB RAM
- Any modern CPU